Scrabble en Java

Alexandre Lou-Poueyou
Aymeric Cassard
Ludovic Andrieu
Jean Mayol
Mathéo Petry
Valentin Cazet

8 avril 2025

Table des matières

1 Règles du jeu **3**
1.1 Règles du jeu 3
1.1.1 Installation de la partie 3
1.1.2 Déroulement de la partie 3
1.1.3 Fin de la partie 4
1.1.4 Comptage des points 4
1.1.5 Variantes du jeu 4

2 L'existant **5**
2.1 Choix du jeu et du langage 5
2.1.1 Pourquoi le Scrabble ? 5
2.1.2 Java 5
2.2 Existant du projet 5

3 Structures de données & algorithmes **7**
3.1 Structures de données 7
3.1.1 `Dictionary.java` 7
3.1.2 `Letter.java` 7
3.1.3 `LetterCounter.java` 7
3.1.4 `Grid.java` 8
3.1.5 `History.java` et `HistNode.java` 8

3.2 Algorithmes 9
3.2.1 `Parser` 9
3.2.2 `Tokens` 10
3.2.3 `Serializer` 13
3.2.4 `placeLetters()` 13

4 Architecture **15**
4.1 Architecture / Design Pattern 15
4.2 Gestion des erreurs 16
4.3 Dépendances du projet 16

5 Performances et limitations **17**

6 Critiques et perspectives **17**

7 Besoins réalisés **18**

8 Bibliographie **19**

1 Règles du jeu

TARGET :
1 Règles du jeu Jean

1.1 Règles du jeu

Le Scrabble est un jeu de société se jouant de 2 à 4 joueurs et qui consiste à former des mots sur une grille carrée de 15×15 cases. Le but du jeu est de marquer le plus de points en posant des mots. Le jeu du Scrabble contient un plateau de jeu, un ensemble de jetons de lettres variant selon le pays, un sac de lettres et des chevalets de jeu permettant de poser les jetons tout en cachant ses lettres aux adversaires.

1.1.1 Installation de la partie

On met tous les jetons de scrabble dans le sac de lettres. On pose au centre de la table le plateau de jeu et devant chaque joueur des chevalets de jeu. Chaque joueur pioche sans regarder une lettre dans le sac de lettres. Celui qui a la lettre la plus haute dans l'alphabet (A > B, B > C, etc.) commence. Si un joueur pioche le joker, il doit re-piocher. Ils remettent toutes les lettres quand le 1er joueur est déterminé.
Le 1er joueur pioche ensuite 7 lettres dans le sac de lettres et les garde sans les montrer aux adversaires. Les autres joueurs font de même dans le sens horaire.
La partie peut désormais commencer.

1.1.2 Déroulement de la partie

Au début de la partie, le 1er joueur pose sur le plateau un mot qu'il peut former à partir des lettres de son chevalet. Une lettre du mot doit être posée sur le centre du plateau (la case de coordonnée $h8$ pour un plateau standard). Il pioche ensuite dans le sac de lettres un nombre de lettres égales au nombre de lettres posé sur le plateau. Il pioche donc jusqu'à avoir 7 lettres sur son chevalet.
Le 2ème joueur fait de même mais doit rattacher son mot au mot du 1er joueur et ne peut superposer ces lettres sur le mot du 1er joueur. Il faut que le mot rattaché au mot du 1er joueur soit aussi un mot légal, c'est a dire un mot appartenant au dictionnaire. Ensuite, il pioche, suivant la même règle que celle du 1er joueur. La partie se déroule ainsi de suite jusqu'à la fin.

Les seules actions que peut entreprendre le joueur lors de son tour sont :

— Poser un mot.

— Échanger des lettres avec la pioche.

— Passer son tour.

1.1.3 Fin de la partie

Les conditions de fin de partie sont les suivantes :

— **Reliquat épuisé** : Un joueur a épuisé toutes ses lettres et la pioche est vide. Les joueurs qui n'ont pas terminés retirent de leurs scores la valeur des lettres qu'ils n'ont pas posés et le joueur qui a finit ajoute a son score l'ensemble de ces valeurs ainsi retirés.

— **Blocage** : Les joueurs ne trouvent plus de mots a jouer et il reste moins de 7 lettres dans la pioche. Les valeurs des lettres de leur mains sont déduites de leur score final.

Le joueur avec le score le plus élevé gagne.

1.1.4 Comptage des points

La grille contient des cases bonus :

— Lettre double : double la valeur de la lettre posée.

— Lettre triple : triple la valeur de la lettre posée.

— Mot double : double la valeur du mot posé.

— Mot triple : triple la valeur du mot posé.

Le score du coup posé est la somme des valeurs des lettres de chaque mot formé par le coup, majorée des points obtenus en plaçant des lettres sur des cases multiplicatrices. On applique les bonus de lettres avant d'appliquer celui des mots. Si plusieurs mots sont formés occupant la même case multiplicatrice d'un seul coup, le joueur obtient le bonus correspondant à chaque mot formé. Une fois qu'une case multiplicatrice a été utilisée, elle ne rapporte plus de points si un autre mot est formé ultérieurement avec la tuile qui l'occupe.

Si un joueur pose toutes ses lettres en un tour, il réalise un "scrabble" et obtient un bonus de 50 points.

1.1.5 Variantes du jeu

Il existe plusieurs variantes au scrabble, dont :

— **Super-scrabble** :
  grille de taille 21×21
  la pioche contient plus de lettres
  chaque lettres a un multiplicateurs de score différents

— **Blitz** :
  chaque joueur dispose d'un temps imparti
  ce temps s'écoule durant son tous
  un joueur n'ayant plus de temps perd la partie

2 L'existant

TARGET :
2 L'existant Jean

2.1 Choix du jeu et du langage

2.1.1 Pourquoi le Scrabble ?

Notre choix pour le projet s'est porté sur le Scrabble. Cela nous a semblé être le meilleur choix pour notre groupe car ce jeu est très répandu et ses règles assez basiques permettent une compréhension facile du sujet. De plus, celles-ci ne possèdent pas de cas particuliers complexes à d'autres jeux comme les échecs ou le Tafl [4], ce qui nous permet de nous concentrer sur des aspects plus techniques de la gestion et création du projet (Génie Logiciel, Interface Graphique, bon déroulement du jeu).

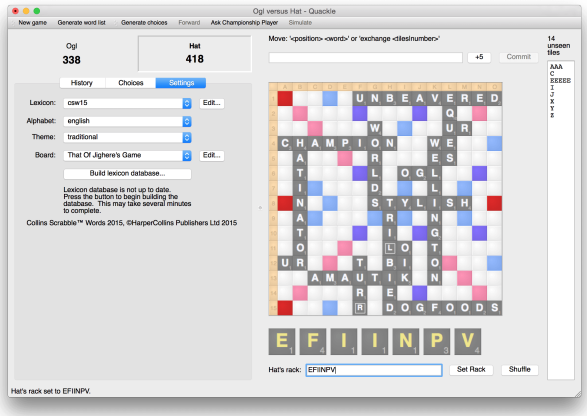
2.1.2 Java

Le langage de programmation ayant retenu notre attention est Java grâce à sa richesse, son accessibilité et sa lisibilité, de par sa documentation [3] très complète et ses nombreuses librairies robustes qui peuvent nous simplifier diverses tâches. De plus, nous avons comparé Java avec les autres langages disponibles (Python, C) et nous avons repéré des points négatifs dans les autres langages :

2.2 Existant du projet

TODO :
reformuler platement + add [?]
BY JEAN

Le scrabble étant un jeu très connu et répandu, il n'est pas étonnant de trouver des existants de ce jeu, principalement pour ce qui est du test de performances d'algorithmes. On peut retrouver dans cette situation le Quackle [1] développé par le MIT et étant une implémentation du scrabble avec une IA utilisant l'algorithme Gaddag pour lui permettre de jouer le meilleur coup possible.



Mais on peut aussi retrouver des implémentations du Scrabble se concentrant sur l'aspect ludique et distrayant via l'ajout de différents modes de jeu, d'un aspect compétitif ou d'éléments de customisation. Par exemple, les jeux de Scrabble développés pour les mobiles.



FIGURE 1 – Scrabble Go sur mobile

3 Structures de données & algorithmes

TARGET :
3 Algorithmes & structures de données Val et collabo
N'utilisez pas une image pour afficher du code, utilisez minted ou lstlisting

3.1 Structures de données

3.1.1 `Dictionary.java`

La classe dictionnaire permet de stocker l'ensemble des mots autorisés pour une partie de Scrabble. Un dictionnaire est un ensemble de mots, un mot existant en un seul exemplaire. Cette classe est implémentée grâce à un `HashSet<Word`. Il s'agit d'un type permettant de créer un ensemble mathématique de Word, c'est-à-dire de mots. Lors d'une partie, il est souvent nécessaire de savoir si un mot appartient au dictionnaire, pour pouvoir vérifier qu'un joueur place un mot existant, un mot dit légal. De ce fait, l'ensemble de mots est de type `HashSet<Word>` car ce type d'ensemble est implémenté comme une table de hachage, ce qui permet de vérifier l'existence d'un mot dans le dictionnaire en une complexité $\mathcal{O}(1)$.

3.1.2 `Letter.java`

La grille du Scrabble est composée de lettres. Une classe `Letter` a été créée pour répondre à ce besoin. La classe `Letter` est wrapper du type **`char`**. Dans le Scrabble, les lettres sont toutes en majuscules. Cette classe permet de créer à partir de caractères en majuscules, des lettres bien formées. Cela permet d'être sûr de tout le temps manipuler la même lettre, et régler le problème de minuscule/-majuscule. De plus, tous les caractères n'étant pas des lettres, cette classe permet aussi d'assurer d'avoir une lettre valide.

3.1.3 `LetterCounter.java`

La classe `LetterCounter` est un wrapper sur une `Hashtable<Letter, Integer>`. Cette classe permet, par exemple, de stocker les lettres du chevalet d'un joueur. Le but de cette classe est de savoir quelle lettre contient le chevalet, ainsi que le nombre d'occurrences. Pour cela, elle est implémentée comme une `Hashtable<Letter, Integer>`. Il s'agit d'une table de hachage, qui pour une lettre donnée renvoie son nombre d'concurrence. Étant donné que cette classe est un wrapper, des méthodes telles que l'ajout d'occurrence, la suppression, le nombre d'occurence, etc. ont été ajouté. Grâce à ces méthodes l'intégrité est garantie.

3.1.4 `Grid.java`

La classe Grid permet de stocker le plateau de jeux.
Elle contient une matrice de lettres placées : `Letter[][] placedLetters`.
Cette classe contient aussi une autre matrice permettant de connaître les multiplicateurs de points en fonction des positions sur la grille :
`Hashtable<GridPos, ScoreMultiplier> multiplicators`. Cette table de hachage permet de savoir pour une `GridPos` (x, y) donnée quel sera le multiplicateur à appliquer.
Les multiplicateurs déjà utilisés sont marqués dans
`boolean[][] multiplicatorsUsed`. En fonction de la langue et de l'activation ou non du mode super Scrabble, chaque lettre vaut un nombre de points différent. Les points associés aux lettres sont stockés dans `LettersPoints pointsPerLetter`.
La taille de la grille est stockée dans **`int`** `size`.

Les attributs complexes ont été expliqués ci-dessus, mais elle en comporte d'autres, voir 2 :

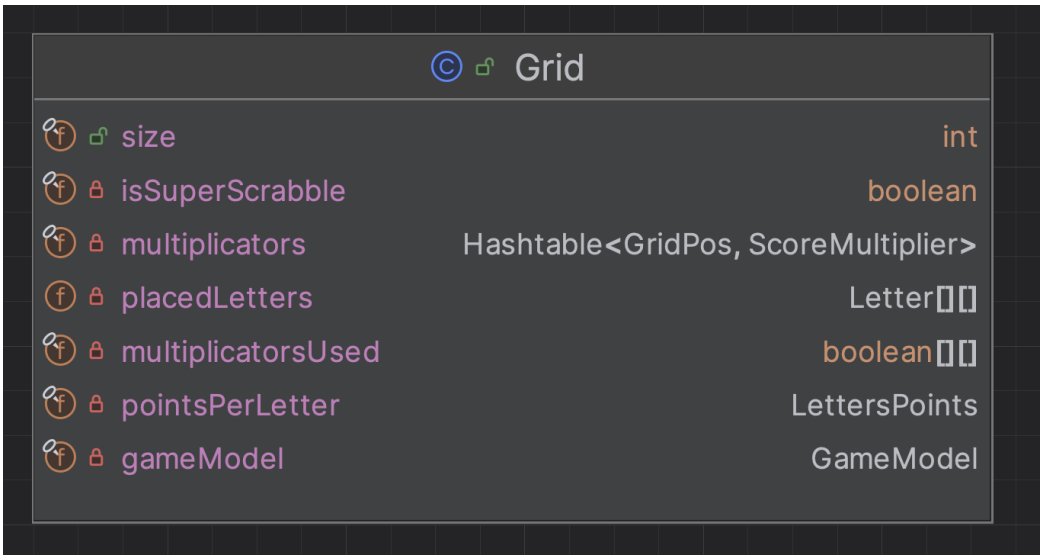


FIGURE 2 – Grid class attributes UML

3.1.5 `History.java` et `HistNode.java`

L'Historique est une pile implementée par une liste doublement chainée classique. Les noeuds de cette liste, `HistNode`, contiennent les donnees necéssaires

pour pouvoir effectuer une action de la partie.

Cette structure s'avère adaptée pour nos besoins relatifs au jeu. En effet, nous ajoutons seulement des éléments à la fin de l'historique, apres que le joueur aie effectue son coup, la pile est donc adaptée. Quand il s'agit de retrouver des elements de l'historique, nous avons trois cas d'usages :

— Défaire un coup(undo) : On retrouve l'element precedent la tete de pile O(1)

— Refaire un coup(redo) : On retrouve l'element suivant la tete de pile O(1)

— Ecrire l'historique dans un fichier : On parcourt la pile de haut en bas O(n)

Cependant, utiliser une pile pour l'historique n'est pas une solution parfaite. A tout moment de la partie, nous avons seulement un chemin de changement pour l'utilisateur. Concrètement, si un utilisateur revient en arrière dans l'historique, puis décide de jouer un coup, tout l'historique suivant original est abandonné. Une solution aurait été d'utiliser un arbre au lieu d'une pile. **TODO : pseudocode undo / redo**

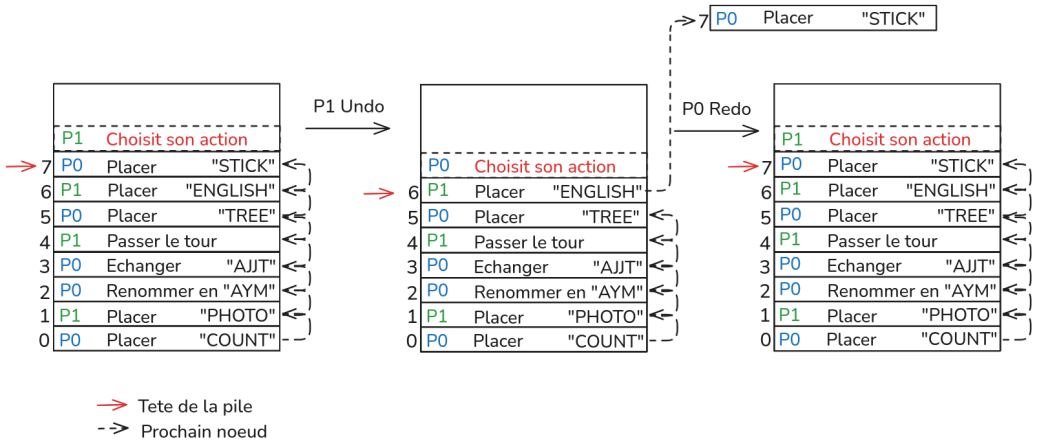


FIGURE 3 – Undo puis redo sur l'historique d'une partie (les références vers le noeud précédent sont omises)

3.2 Algorithmes

3.2.1 `Parser`

L'objectif du parser est d'analyser un fichier de sauvegarde du scrabble et d'en sortir un modèle utilisable, tout en jetant des erreurs indiquant la ligne concernée a l'utilisateur. Pour cela nous avons implémenté notre propre solution de parsing.

TODO : schéma parsing string

3.2.2 `Tokens`

Notre parseur ne traite pas directement de charactères ou de lignes du fichier d'entrée, il agit sur des elements abstraits, des tokens. Ces tokens représentent une valeur "terminale" du fichier de sauvegarde, c'est a dire une valeur qui ne peut pas être subdivisée en éléments plus petits. Cette abstraction permet de diminuer la complexite des algorithmes du parseur, car nous n'avons pas besoin de nous soucier du placement exact des caractères dans le fichier pour en extraire le contenu.

Liste des Tokens		
Nom	Description	Exemple
Ponctuation	Un de "`.{}<>[],;`"	;
Opérateur	=	=
Nombre	1 a 8 chiffres	30
Nombre Négatif	1 a 8 chiffres, précédé de "-"	-30
Long	9+ chiffres	111111111
Long Négatif	9+ chiffres, precede de "-"	-111111111
Identifiant	`[A-Z][0-z]+`	Language
String	"`.+`"	"mcts"
Move	`[a-z][0-9]+[a-z]`	h8h

Note : Les regex sont seulement inclus pour être concis, ils ne sont pas utilisés pour parser les tokens dans le code. Il est possible que .+ aie des divergences avec notre implémentation en Java.

Nous pouvons donc facilement décrire la série de token souhaitée au parseur, puis former un modèle à partir de ces valeurs.

Les méthodes finales de l'algorithme de parsing 1 ne sont pas développées car peu pertinenentes d'un point de vue algorithmique. Il est cependant notable que nous sérialisons très peu d'informations dans ce modele. En effet, au lieu de récupérer les états complets de la partie depuis le fichier, nous récupérons l'état initial au début de la partie, ainsi qu'une liste des coups, que nous rejouons 1 à 1 pour revenir à l'état décrit par le fichier. Cette manière de retrouver les états est possible car nous invoquons tous nos appels aléatoires (pioche) à l'aide de l'algorithme pseudorandom de Java instancié avec une Seed, qui elle est sérialisée. Cette seed est retrouvée, parmi d'autres propriétés dans `parserVarCourante()` et les coups sont rejoués lors de `parserTourCourant()`.

Algorithme 1 : Parsing du modèle serializé

Entrées : $tokenizer$
Sorties : $modele$
Fonction `EstIdentifiant(`$String$`)` **:**
  $prochainToken \leftarrow tokenizer.prochain()$
  Retourner prochainToken.type est "Identifiant" et
   prochainToken.value est String
Fonction `EstCleGame(`$String$`)` **:**
  $prochainToken \leftarrow tokenizer.prochain()$
  Retourner prochainToken.type est "Identifiant" et
   prochainToken.value $\in$ clesGame
Fonction `EstIdAction(`$String$`)` **:**
  $prochainToken \leftarrow tokenizer.prochain()$
  Retourner prochainToken.type est "Identifiant" et
   prochainToken.value $\in$ typesAction
Fonction `Parser(`$tokenizer$`)` **:**
  $prochainToken \leftarrow tokenizer.prochain()$
  tant que $prochainToken \neq EOF$ **faire**
    PasserPonctuation('<')
    si *EstIdentifiant("GAME")* **alors**
      tant que *EstCleGame()* **faire**
        passerOperateur()
        $modele \leftarrow parserVarCourante()$
    si *EstIdentifiant("HISTORY")* **alors**
      $prochainToken \leftarrow tokenizer.prochain()$
      tant que *prochainToken.type est "Nombre"* **faire**
        PasserPonctuation('.')
        si *EstIdAction()* **alors**
          $modele \leftarrow parserTourCourant()$
        PasserPonctuation(' ;')
    PasserPonctuation('>')
  retourner $modele$

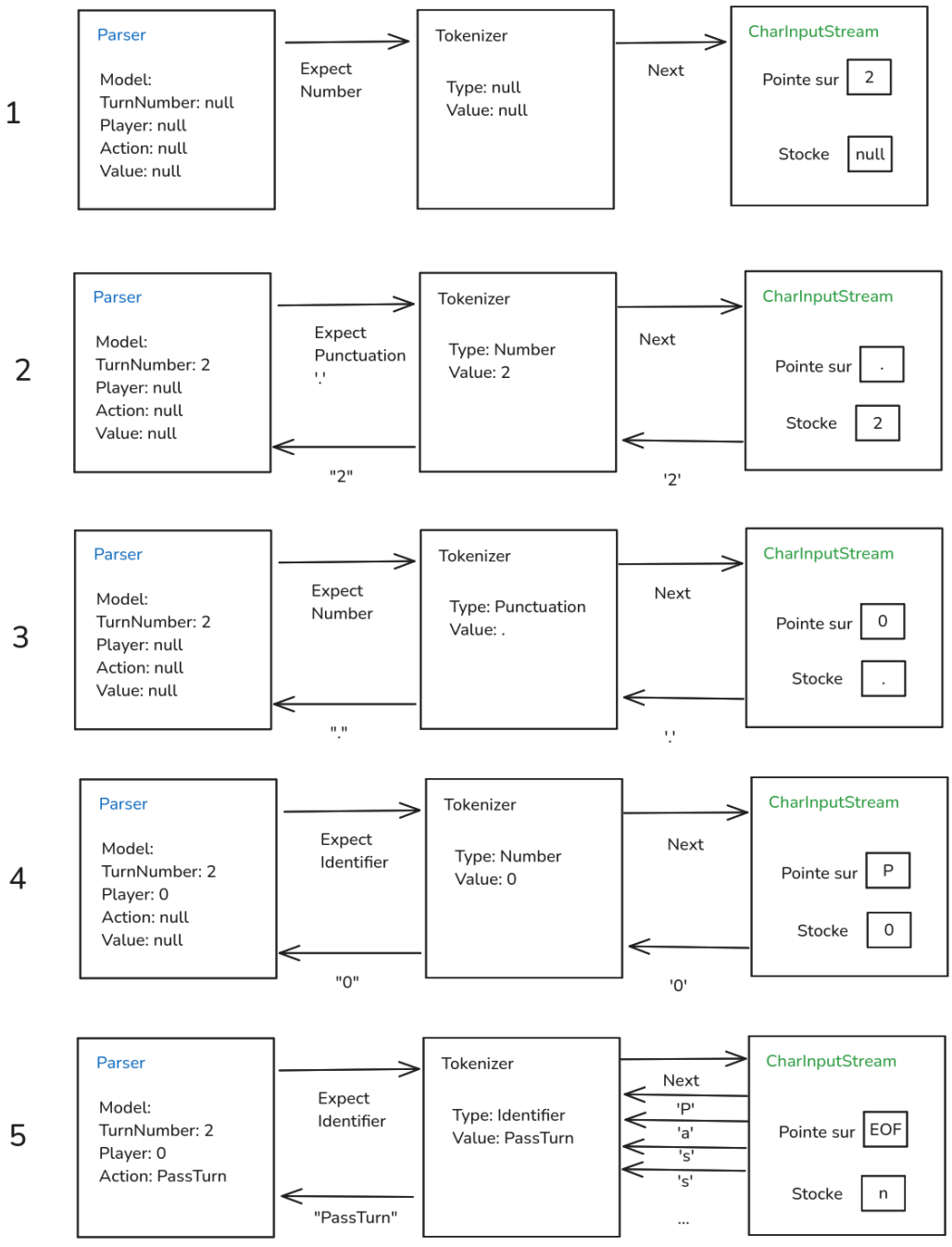


FIGURE 4 – Schema du parsing du mot "2. 0 PassTurn"

3.2.3 `Serializer`

Le Serializer est la classe qui s'occupe de sauvegarder l'état d'une partie dans un fichier ainsi que de le recharger. Pour ce qui est de charger l'état de la partie, elle appelle le parseur décrit précédemment. Quand il s'agit de sauvegarder un modèle vers un fichier, on utilise notre structure de donnée `History`.

Algorithme 2 : Sérialisation de l'historique vers String

Entrées : $history$
Sorties : $texteSerialise$
$noeud \leftarrow history.currentNode$
$pileActions \leftarrow []$
tant que $noeud \neq debut$ **faire**
    pileActions.empiler(noeud.action)
    $noeud \leftarrow noeud.precedent$
pour tout $action \in pileActions$
    $texteSerialise \leftarrow action.stringSerialise$
retourner $texteSerialise$

Bien que notre sérialisation nous coûte O(n) opérations où n est le nombre d'actions jouées dans la partie, on constate que l'on itère tout de même deux fois sur les actions. Une fois pour récupérer les actions dans l'ordre, une deuxième fois pour produire le string. Nous adoptons cette approche car cela nous permet de créer le String dans l'ordre d'arrivée des actions, en effet itérer une seule fois et recupérer les actions dans l'ordre inverse nous demanderait de sérialiser en ajoutant à la position 0 du string, ce qui est une opération coûteuse. **TODO AYM Serializer AYM**

3.2.4 `placeLetters()`

Explication en pseudo-code de haut niveau de la méthode
`public List<OwnedWord> placeLetters(PlaceWordAction pwa)`
de `Grid.class` liée à la `Feature 32` :

Algorithme 3 : List<OwnedWord> placeLetters(PlaceWordAction)

Data : PlaceWordAction {
| tableau de Letter a placer sur la grille
| ligne x et colonne y où placer les lettres
| direction de placement
}
Result : List<OwnedWord> {
| liste de mots formés avec leurs positions sur la grille
}
Vérification des prérequis :
| Si aucune lettre n'est à placer, renvoyer une erreur
| Si la position de départ n'est pas dans les limites de la grille, renvoyer une erreur
| Si la case de départ n'est pas vide, lever une erreur
Création des variables :
| Liste de mots résultants formés
| Liste pour stocker les futurs positions des lettres à placer
| Direction du mot à placer et sa direction opposée
Calculer les positions des futurs lettres à placer :
| Pour chaque lettre à placer, calculer et enregistrer sa futur position;
`/* Sert quand des lettres sont déjà présentes sur la grille */`
Construire le mot principal :
| Déterminer les positions de début et de fin du mot principal et le construire
Construire les mots auxiliaires :
| Pour chaque lettre, vérifier si elle forme un mot avec les lettres adjacentes et le construire si c'est le cas
Vérifier la validité des mots :
| S'assurer que tous les mots construits sont présents dans le dictionnaire, sinon renvoyer une erreur
Contraintes de placements :
| Vérifier la règle du premier placement sur la grille :
| | Si c'est le premier placement sur la grille de la partie, vérifier qu'il touche le centre
| Vérifier les placements adjacents :
| | S'assurer que les mots sont placés à côté d'autres mots
Placer les lettres sur la grille :
| Pour chaque lettre, placer la lettre à sa position calculée
Retourner la liste des mots construits :
| Retourner la liste des mots qui ont été placés sur la grille

4 Architecture

TARGET :
4 Architecture Val

4.1 Architecture / Design Pattern

Nous avons choisi une architecture`Model-View-Presenter`(MVP [5]) pour organiser les modules du projet. Cette architecture a été choisie car elle pose les lignes directrices sur la gestion des flux entre les classes. De plus, elle est optimisée pour la gestion d'affichage graphique multiple.

Le`Model`contient les différentes structures de données, et autres éléments logiques relatifs aux contraintes du jeu. Les éléments de cette couche correspondent à ce qui est nécessaire à la Game (joueurs, lettres, multiplicateur, etc).

La `View` est constituée de tous les éléments visibles par l'utilisateur. Ici, elle est découpée en deux. La partie `CLI`, et la partie `GUI`. L'architecture MVP est très pratique dans ce cas là. Nous possédons deux moyens d'affichage, qui sont soit dans la console, soit en graphique. Pour cela nous pouvons simplement choisir à qui le `Presenter` enverra les données pour qu'elle puisse être affichée par la `View`, que ça soit la `ViewCLI` ou `ViewGUI`.

Le `Presenter` reçoit les demandes de l'utilisateur depuis la `View`, les traite, demande les modifications au `Model`, puis récupère les nouvelles informations contenues dans le `Model`, et donne ces nouvelles informations actualisées à la `View`.

Pour la gestion des flux, voir 5 :

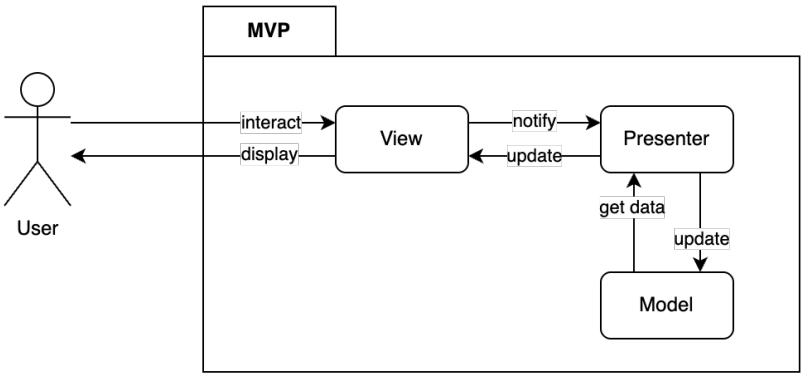


FIGURE 5 – MVP scheme

4.2 Gestion des erreurs

Chaque classe, ainsi que ses méthodes, renvoie des `CustomError` lors de problèmes rencontrés. Toutes les classes sont appelées par le `Presenter`qui les coordonne. Le `Presenter`contient la `game loop`qui s'occupe de la gestion du jeu, et qui récupère les erreurs soulevées de toutes les classes du programme. Lorsqu'une erreur est soulevée, il l'affiche à l'utilisateur. De ce fait, toute erreur pouvant arriver lors de l'exécution sera rattrapée par la `game loop`et afficher, ne provoquant pas un arrêt brutal du programme.

4.3 Dépendances du projet

Le projet est codé en Java version `17.0.13`, et est compilé à l'aide du système de build Maven. Afin de respecter les spécifications du projet, un certain nombre de dépendances sont ajoutées :

— `JUnit` : Framework de test, utilisé pour les différents scénarios de test du code.

— `Jacoco` : Librairie de couverture de code, utilisée en conjonction avec JUnit pour certifier le pourcentage de code source testé.

— `Commons-cli` : API d'analyse d'options de commandes. Utilisé pour interpréter les options passées en ligne de commande.

— `JavaFx` : Framework d'interface utilisateur, utilisé pour la partie interface graphique.

— `Spotless` : Outil d'analyse de code permettant de vérifier la conformité du formatage avec le style de codage de Google.

5 Performances et limitations

Le projet Scrabble a été élaboré en se focalisant sur la clarté, la modularité et la facilité de maintenance du code, plutôt que sur une optimisation précise de la performance. Cette décision est totalement assumée et en adéquation avec les buts éducatifs du projet. Le langage Java, utilisé pour l'implémentation, propose des abstractions de haut niveau et une gestion automatique de la mémoire grâce à un garbage collector. Cela nous a permis de focaliser nos efforts sur l'implémentation du jeu et des règles du Scrabble ainsi que sur la solidité du fonctionnement global.

En pratique, les performances observées sont très satisfaisantes. Le jeu se charge quasiment instantanément sur des machines ordinaires, qu'il soit utilisé via une interface en ligne de commande (CLI) ou une interface graphique (GUI). Aucune action, y compris celles qui se rapportent au chargement du dictionnaire officiel du Scrabble (en français et en anglais), à la vérification des mots, à l'évaluation des points ou à la disposition des lettres sur le plateau, n'a montré de délai perceptible. Il convient de noter que, bien qu'aucun instrument de mesure de performance n'ait été intégré dans l'application (tels que des chronomètres ou des analyseurs de performance), l'ensemble du comportement a été évalué empiriquement à travers les interactions utilisateur : les réactions de l'interface sont immédiates, sans latence visible.

Du point de vue de l'interface, la CLI est stable, réactive et complète. Du côté de la GUI, tout est stable et fonctionnel. Aucune fonctionnalité n'a été négligée lors de l'intégration des fonctionnalités de la CLI dans la GUI.

En termes de limitation, le projet remplit en grande partie les fonctionnalités spécifiées dans le cahier des charges initial. Seule la partie sur l'intelligence artificielle et l'implémentation d'un joueur artificiel n'a pas été implémentée. Cela représente une piste d'amélioration future, hors du périmètre de cette version. Aucune limitation technique n'empêche a priori l'évolution du code vers cette fonctionnalité.

6 Critiques et perspectives

TARGET :
6 Critiques et perspectives Alex Matheo

TODO :

Jean vous a deja ecrit un peu, mais c'est pas suffisant. Corriger le fait qu'il a ecrit comme si il parlait a l'oral, rajouter beaucoup de texte. 1 page mini pour ces deux sections. Ne descendez pas le projet ! Ex de Jean "On a pris trop a la légère certaines parties du projet", ne surtout pas ecrire comme ca, mais plutot, "nous avons sous-estime le temps de dev sur certaines parties du projet" etc. De plus, le texte de jean etait pour une autre section, donc il faut presque tout rephraser.
Si vous avez pas d'inspi sur les "critiques" de notre projet, regarder ce qu'on a pas dev du cahier des charges

Grâce à une gestion et vision précise du projet réalisé par un de nos membres, nous ne nous sommes pas perdus sur l'architecture du projet. En revanche il est arrivé que nous ayons dû faire des choix et des modifications soudaines de cette architecture au fur et à mesure de l'avancement du projet. De plus certaines parties du projet aurait sûrement pu être plus fracturer afin qu'il soit plus lisible et compréhensible. Prendre plus de temps, faire des points pour restructurer si nécessaires semble être bénéfique pour un projet de cette envergure.

Le projet bien qu'il marche et qu'on peut y jouer normalement, manque de plusieurs fonctionnalités tels que l'implémentation d'une l'IA. On peut jouer directement depuis la console et on a une interface graphique fonctionnelle. On est malheureusement assez limités sur certaines fonctionnalités qu'on aurait voulu améliorer mais qui sont compliqués a mettre en place et de plus ne sont pas forcément prioritaire. Par exemple le mode multijoueur sur plusieurs PC, un système de classement en fonction du score le plus élevé. On a pris trop a la légère certaines parties du projet et avons pris ainsi du retard qui s'est répercutés à la fin du projet. Nous avons donc choisi de prioriser un code propre et efficace, sans bug que nous précipiter pour réaliser le plus de fonctionnalités. On a manqués d'anticipations et nous devrons être plus vigilent a cela. Malgré tout, l'implémentation d'une IA et de multiples fonctionnalités afin de perfectionner le projet sont les axes principaux d'améliorations futurs d'un projet tel quel.

7 Besoins réalisés

TARGET :
7 Besoins réalisés

F1	✓	F21	✓	F41	✓
F2	✓	F22	✓	F42	✓
F3	✓	F23	✓	F43	✓
F4	✓	F24	✓	F44	✓
F5	W15	F25	✓	F45	✓
F6	W15	F26	?	F46	✓
F7	✓	F27	W15	F47	×
F8	W15	F28	✓	F48	Alex, Mathéo, W15
F9	✓	F29	Alex, Mathéo, W15	F49	Alex, Mathéo, W15
F10	✓	F30	✓	F50	Alex, Mathéo, W15
F11	✓	F31	✓	F51	✓
F12	✓	F32	✓	F52	Alex, Mathéo, W15
F13	✓	F33	✓	F53	Alex, Mathéo, W15
F14	✓	F34	✓	F54	×
F15	✓	F35	✓	F55	×
F16	✓	F36	✓	F56	×
F17	✓	F37	✓	F57	×
F18	✓	F38	✓	F58	×
F19	✓	F39	✓	F59	×
F20	✓	F40	✓	F60	×

8 Bibliographie

TARGET :
8 Bibliographie (>10 références)

TODO :
bof la citation du Tafl

La correction linguistique a été effectuée à l'aide de ChatGPT d'OpenAI [2].

Références

[1] A. W. A. . G. J. Jacobson. The world's fastest scrabble program. *Programming Techniques and Data Structures*, 31 :572–578, May 1988.

[2] OpenAI. Correction linguistique effectuée à l'aide de ChatGPT. `https://www.openai.com/chatgpt`, 2025. Correction linguistique réalisée à l'aide de l'outil ChatGPT d'OpenAI.

[3] Oracle and/or its affiliates. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/docs/api/index.html, 2024.

[4] Peuples nordiques. Tafl, Probablement entre le 4ème et le 12ème siècle. Ancien jeu de stratégie, originaire des cultures nordiques.

[5] Wikipedia. Model–View–Presenter. https://en.wikipedia.org/wiki/ModelâĂŞviewâĂŞpresenter, 2024.